Assignment 2: Convolution

Curtis Meister

Contents

```
library(tidyverse)
library(fs)
library(keras)
library(tensorflow)

set.seed(123)
stopifnot(is_keras_available())

base_images_dir <- path_expand("~/Desktop/PetImages")
work_dir <- path_expand("~/Desktop/assignment2_runs")
dir_create(work_dir)

stopifnot(dir_exists(path(base_images_dir, "Cat")))
stopifnot(dir_exists(path(base_images_dir, "Dog")))
```

```
library(reticulate)
pil <- import("PIL.Image", delay_load = TRUE)

is_readable_image <- function(fp) {
  ok <- TRUE
  tryCatch({
    img <- pil$open(fp)
    img$load()
  }, error = function(e) {
    ok <<- FALSE
  })
  ok
}

remove_corrupt_images <- function(class_dir) {
  files <- list.files(class_dir, pattern = "(?i)\\.jpg$", full.names = TRUE)
  bad <- c()
  for (f in files) {
    if (!is_readable_image(f)) bad <- c(bad, f)
  }
  if (length(bad) > 0) {
    file.remove(bad)
  }
  tibble(dir = class_dir, scanned = length(files), removed = length(bad))
}
```

```r
cat_dir <- path_expand("~/Desktop/PetImages/Cat")
dog_dir <- path_expand("~/Desktop/PetImages/Dog")

clean_summary <- bind_rows(
  remove_corrupt_images(cat_dir),
  remove_corrupt_images(dog_dir)
)

clean_summary
```

```r
list_class_images <- function(class_dir) {
  files <- dir_ls(class_dir, regexp = "(?i)\\.jpg$", recurse = FALSE, type = "file")
  as.character(files)
}

create_split_tree <- function(split_root) {
  dir_create(path(split_root, "train", "cats"), recurse = TRUE)
  dir_create(path(split_root, "train", "dogs"), recurse = TRUE)
  dir_create(path(split_root, "validation", "cats"), recurse = TRUE)
  dir_create(path(split_root, "validation", "dogs"), recurse = TRUE)
  dir_create(path(split_root, "test", "cats"), recurse = TRUE)
  dir_create(path(split_root, "test", "dogs"), recurse = TRUE)
}

copy_into_split <- function(files, dest_dir) {
  dir_create(dest_dir, recurse = TRUE)
  file_copy(files, dest_dir, overwrite = TRUE)
}

make_splits <- function(train_n, val_n = 500, test_n = 500, seed = 123, split_name) {
  set.seed(seed)

  cats <- list_class_images(path(base_images_dir, "Cat"))
  dogs <- list_class_images(path(base_images_dir, "Dog"))

  cats <- sample(cats)
  dogs <- sample(dogs)

  needed_per_class <- train_n + val_n + test_n
  stopifnot(length(cats) >= needed_per_class, length(dogs) >= needed_per_class)

  cats_train <- cats[1:train_n]
  cats_val   <- cats[(train_n + 1):(train_n + val_n)]
  cats_test  <- cats[(train_n + val_n + 1):(train_n + val_n + test_n)]

  dogs_train <- dogs[1:train_n]
  dogs_val   <- dogs[(train_n + 1):(train_n + val_n)]
  dogs_test  <- dogs[(train_n + val_n + 1):(train_n + val_n + test_n)]

  split_root <- path(work_dir, split_name)
  if (dir_exists(split_root)) dir_delete(split_root)
  create_split_tree(split_root)
```

```
  copy_into_split(cats_train, path(split_root, "train", "cats"))
  copy_into_split(dogs_train, path(split_root, "train", "dogs"))
  copy_into_split(cats_val,   path(split_root, "validation", "cats"))
  copy_into_split(dogs_val,   path(split_root, "validation", "dogs"))
  copy_into_split(cats_test,  path(split_root, "test", "cats"))
  copy_into_split(dogs_test,  path(split_root, "test", "dogs"))

  split_root
}
```

```
img_size <- c(150, 150)
batch_size <- 20

make_generators <- function(split_root, augment = TRUE) {
  if (augment) {
    train_datagen <- image_data_generator(
      rescale = 1/255,
      rotation_range = 40,
      width_shift_range = 0.2,
      height_shift_range = 0.2,
      shear_range = 0.2,
      zoom_range = 0.2,
      horizontal_flip = TRUE,
      fill_mode = "nearest"
    )
  } else {
    train_datagen <- image_data_generator(rescale = 1/255)
  }

  test_datagen <- image_data_generator(rescale = 1/255)

  train_gen <- flow_images_from_directory(
    path(split_root, "train"),
    train_datagen,
    target_size = img_size,
    batch_size = batch_size,
    class_mode = "binary",
    classes = c("cats","dogs")
  )

  val_gen <- flow_images_from_directory(
    path(split_root, "validation"),
    test_datagen,
    target_size = img_size,
    batch_size = batch_size,
    class_mode = "binary",
    classes = c("cats","dogs")
  )

  test_gen <- flow_images_from_directory(
    path(split_root, "test"),
    test_datagen,
    target_size = img_size,
```

```
    batch_size = batch_size,
    class_mode = "binary",
    shuffle = FALSE,
    classes = c("cats","dogs")
  )

  list(train = train_gen, val = val_gen, test = test_gen)
}
```

```
build_scratch_model <- function() {
  model <- keras_model_sequential() %>%
    layer_conv_2d(filters = 32, kernel_size = c(3,3), activation = "relu",
                  input_shape = c(img_size[1], img_size[2], 3)) %>%
    layer_max_pooling_2d(pool_size = c(2,2)) %>%

    layer_conv_2d(filters = 64, kernel_size = c(3,3), activation = "relu") %>%
    layer_max_pooling_2d(pool_size = c(2,2)) %>%

    layer_conv_2d(filters = 128, kernel_size = c(3,3), activation = "relu") %>%
    layer_max_pooling_2d(pool_size = c(2,2)) %>%

    layer_conv_2d(filters = 128, kernel_size = c(3,3), activation = "relu") %>%
    layer_max_pooling_2d(pool_size = c(2,2)) %>%

    layer_flatten() %>%
    layer_dense(units = 512, activation = "relu") %>%
    layer_dropout(rate = 0.5) %>%
    layer_dense(units = 1, activation = "sigmoid")

  model %>% compile(
    loss = "binary_crossentropy",
    optimizer = optimizer_rmsprop(learning_rate = 1e-4),
    metrics = c("accuracy")
  )

  model
}
```

```
build_vgg16_feature_extractor <- function() {
  conv_base <- application_vgg16(
    weights = "imagenet",
    include_top = FALSE,
    input_shape = c(img_size[1], img_size[2], 3)
  )

  freeze_weights(conv_base)

  model <- keras_model_sequential() %>%
    conv_base %>%
    layer_flatten() %>%
    layer_dense(256, activation = "relu") %>%
    layer_dropout(0.5) %>%
    layer_dense(1, activation = "sigmoid")
```

```
  model %>% compile(
    loss = "binary_crossentropy",
    optimizer = optimizer_rmsprop(learning_rate = 2e-4),
    metrics = c("accuracy")
  )

  model
}
```

```
run_experiment <- function(approach = c("scratch","pretrained"),
                           train_n,
                           epochs = 20,
                           seed = 123) {
  approach <- match.arg(approach)

  split_name <- paste0(approach, "_train", train_n)
  split_root <- make_splits(train_n = train_n, val_n = 500, test_n = 500,
                            seed = seed, split_name = split_name)

  gens <- make_generators(split_root, augment = TRUE)

  model <- if (approach == "scratch") build_scratch_model() else build_vgg16_feature_extractor()

  steps_per_epoch <- as.integer((2 * train_n) / batch_size)
  val_steps <- as.integer((2 * 500) / batch_size)
  test_steps <- as.integer((2 * 500) / batch_size)

  callbacks <- list(
  callback_early_stopping(
    monitor = "val_accuracy",
    patience = 3,
    restore_best_weights = TRUE
    )
)

history <- model %>% fit(
  gens$train,
  steps_per_epoch = steps_per_epoch,
  epochs = epochs,
  validation_data = gens$val,
  validation_steps = val_steps,
  callbacks = callbacks,
  verbose = 0
)

  val_metrics <- model %>% evaluate(gens$val, steps = val_steps, verbose = 0)
  test_metrics <- model %>% evaluate(gens$test, steps = test_steps, verbose = 0)

  tibble(
    approach = approach,
    train_n = train_n,
    epochs = epochs,
    val_loss = as.numeric(val_metrics["loss"]),
```

```
    val_acc  = as.numeric(val_metrics["accuracy"]),
    test_loss = as.numeric(test_metrics["loss"]),
    test_acc  = as.numeric(test_metrics["accuracy"])
  )
}
```

```
scratch_sizes <- c(1000, 1500, 2000)
pretrained_sizes <- c(1000, 500, 1500)

results <- tibble()

for (n in scratch_sizes) {
  message("Scratch model, train_n = ", n)
  one <- run_experiment("scratch", train_n = n, epochs = 12, seed = 123)
  results <- bind_rows(results, one)
}
```

```
## Found 2000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.

## Found 3000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.

## Found 4000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
```

```
for (n in pretrained_sizes) {
  message("Pretrained model, train_n = ", n)
  one <- run_experiment("pretrained", train_n = n, epochs = 6, seed = 123)
  results <- bind_rows(results, one)
}
```

```
## Found 2000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.

## Found 1000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.

## Found 3000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
## Found 1000 images belonging to 2 classes.
```

```
results
```

```
## # A tibble: 6 x 7
##   approach   train_n epochs val_loss val_acc test_loss test_acc
```

```
##   <chr>        <dbl>  <dbl>    <dbl>   <dbl>     <dbl>    <dbl>
## 1 scratch       1000     12    0.597   0.671     0.587    0.682
## 2 scratch       1500     12    0.541   0.742     0.560    0.716
## 3 scratch       2000     12    0.553   0.720     0.558    0.721
## 4 pretrained    1000      6    0.261   0.885     0.240    0.905
## 5 pretrained     500      6    0.273   0.886     0.274    0.886
## 6 pretrained    1500      6    0.239   0.904     0.258    0.892
```

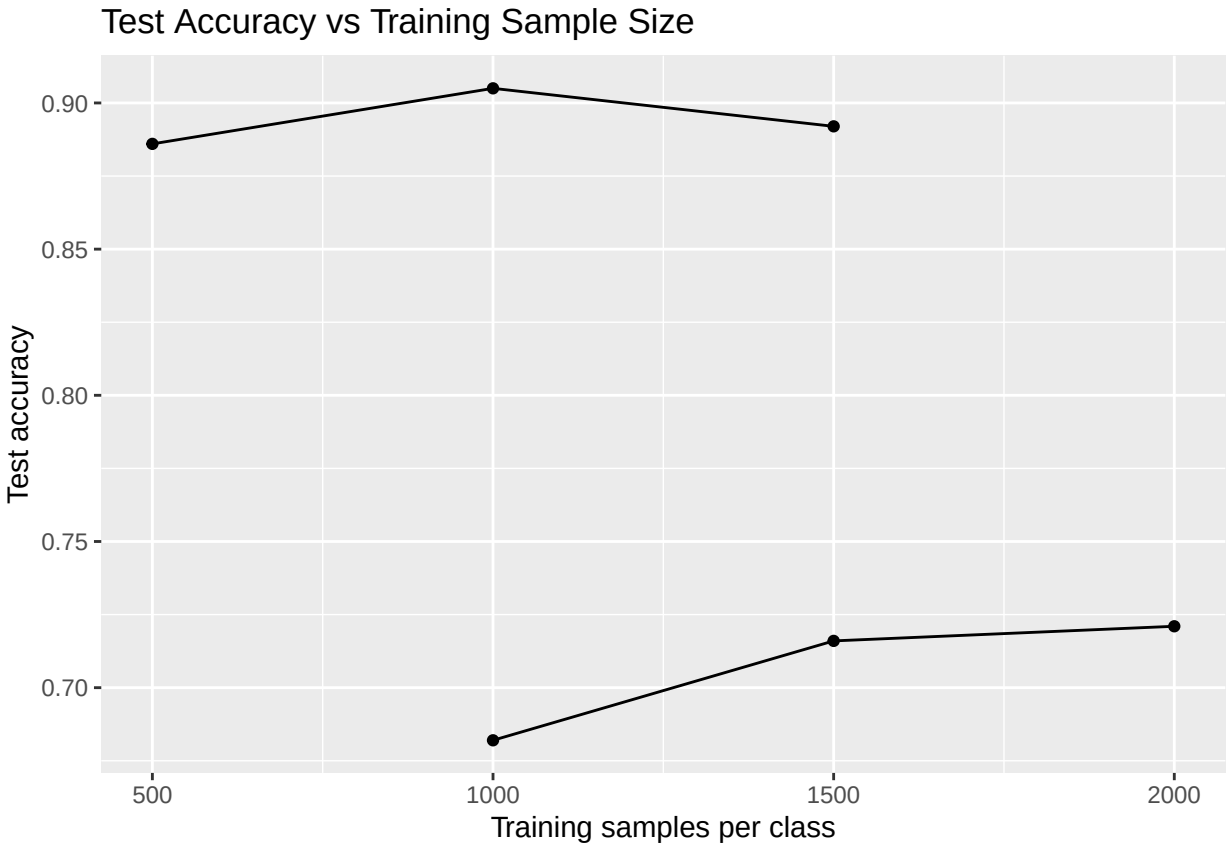
```r
best_by_approach <- results %>%
  group_by(approach) %>%
  slice_max(val_acc, n = 1, with_ties = FALSE)

best_by_approach
```

```
## # A tibble: 2 x 7
## # Groups:   approach [2]
##   approach   train_n epochs val_loss val_acc test_loss test_acc
##   <chr>        <dbl>  <dbl>    <dbl>   <dbl>     <dbl>    <dbl>
## 1 pretrained    1500      6    0.239   0.904     0.258    0.892
## 2 scratch       1500     12    0.541   0.742     0.560    0.716
```

```r
results %>%
  arrange(approach, train_n) %>%
  mutate(val_acc = round(val_acc, 4), test_acc = round(test_acc, 4)) %>%
  knitr::kable()
```

approach	train_n	epochs	val_loss	val_acc	test_loss	test_acc
pretrained	500	6	0.2727180	0.886	0.2736089	0.886
pretrained	1000	6	0.2610338	0.885	0.2396473	0.905
pretrained	1500	6	0.2385759	0.904	0.2576477	0.892
scratch	1000	12	0.5973449	0.671	0.5870206	0.682
scratch	1500	12	0.5405011	0.742	0.5597976	0.716
scratch	2000	12	0.5526725	0.720	0.5578179	0.721

```r
ggplot(results, aes(x = train_n, y = test_acc, group = approach)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Test Accuracy vs Training Sample Size",
    x = "Training samples per class",
    y = "Test accuracy"
  )
```


Test Accuracy vs Training Sample Size
0.90
0.85
0.80
0.75
0.70
Test accuracy
500
1000
1500
2000
Training samples per class